



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P.O.O : **Java**



Chapitre

4

Classe et Objet en **Java**

Sofiane HACHICHA
ISIMG 2025 - 2026

4- Classe et Objet en Java

4.1 Classe en Java (1/3)

P.O.O : Java



- Les classes constituent le concept de base de la programmation orientée objet.
 - Elles permettent de définir des nouveaux types de données qui doivent se comporter comme des types prédéfinis.
- Une classe est introduite par le mot clé **class**.
- Une classe définit généralement deux choses :
 - les données associées aux objets de la classe appelées champs ou **attributs**,
 - les services que peuvent rendre les objets de cette classe sont les **méthodes** définies dans la classe.
- Un **package** (ou paquetage) est une unité (un répertoire) regroupant des classes.





4- Classe et Objet en Java

4.1 Classe en Java (2/3)

Syntaxe générale d'une classe

```
package packagename ;  
  
modificateur_visibilité class nom_Class {  
    modificateur_visibilité data-type attribut1;  
    modificateur_visibilité data-type attribut2;  
    ...  
    modificateur_visibilité data-type attributN;  
  
    modificateur_visibilité Return-type nom_Method1 (paramètres)  
    { // instructions; }  
    ...  
    modificateur_visibilité Return-type nom_Method2 (paramètres)  
    { // instructions; }  
}
```





4- Classe et Objet en Java

4.1 Classe en Java (3/3)

P.O.O : Java



Exemple de classe :

Fichier " TestDate.java "

```
package tn.isimg.cp2i;
class TestDate {
    private int j;    // jour
    private int m;    // mois
    private int a;    // an

    public void initialise(int jour, int mois, int an)
    { j=jour; m=mois; a=an; }

    public void affiche( )
    { System.out.println(j + "/" + m + "/" + a); }
}
```



4- Classe et Objet en Java

4.2 Packages et protection des données (1/8)

P.O.O : Java



● Regroupement de classes

- les packages permettent de regrouper un ensemble de classes portant sur un même thème (classes d'entrées sorties, classes réseau, classes géométriques).
- Un package contient des classes et des packages ce qui permet de constituer une arborescence de packages.
- Les packages permettent une manipulation aisée des groupes de classes, une limitation des espaces de noms et un niveau de protection.

● Constitution d'un paquetage

- dans un fichier **xxx.java** la déclaration **package truc** au début du texte source indique au compilateur que les classes contenues dans le fichier doivent être rangées dans le package **truc**.



4- Classe et Objet en Java

4.2 Packages et protection des données (2/8)

POO : Java



● Nom d'un package :

- Les packages étant organisés en arborescences, la désignation d'un package s'effectue en donnant le chemin sous la forme pointée.
- `bidule.machin.truc` désigne le package `truc` situé dans le package `machin`, lui-même situé dans le package `bidule`.

● Accès aux éléments d'un package :

- Une classe appartenant à un package a un accès à chaque élément du package.
- Une classe peut faire référence à une classe d'un autre package en donnant le chemin complet des packages jusqu'à elle

```
pac1.pac2.classe1 c1;
```

4- Classe et Objet en Java

4.2 Packages et protection des données (3/8)

POO : Java



● Importation des package :

- Une classe peut importer une classe d'un autre package en donnant seulement le nom de la classe et le chemin d'accès à l'aide de la déclaration **import** au début du fichier `xxx.java`.

```
// import de la classe classe1 du package machin.truc
```

```
import machin.truc.classe1;
```

```
// importe toutes les classes du package machin.truc
```

```
import machin.truc.*;
```



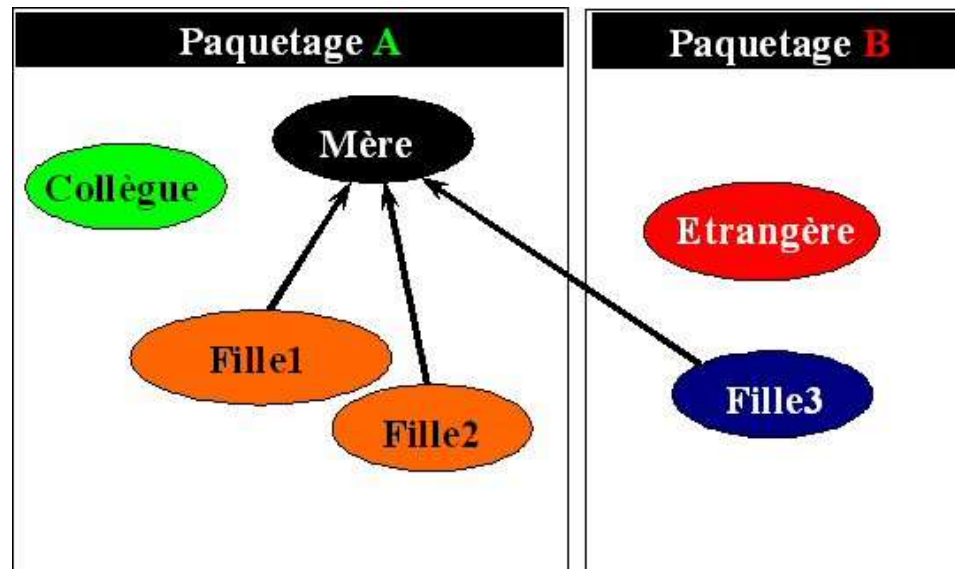
4- Classe et Objet en Java

4.2 Packages et protection des données (4/8)

POO : Java



● Situations possibles des différentes classes :



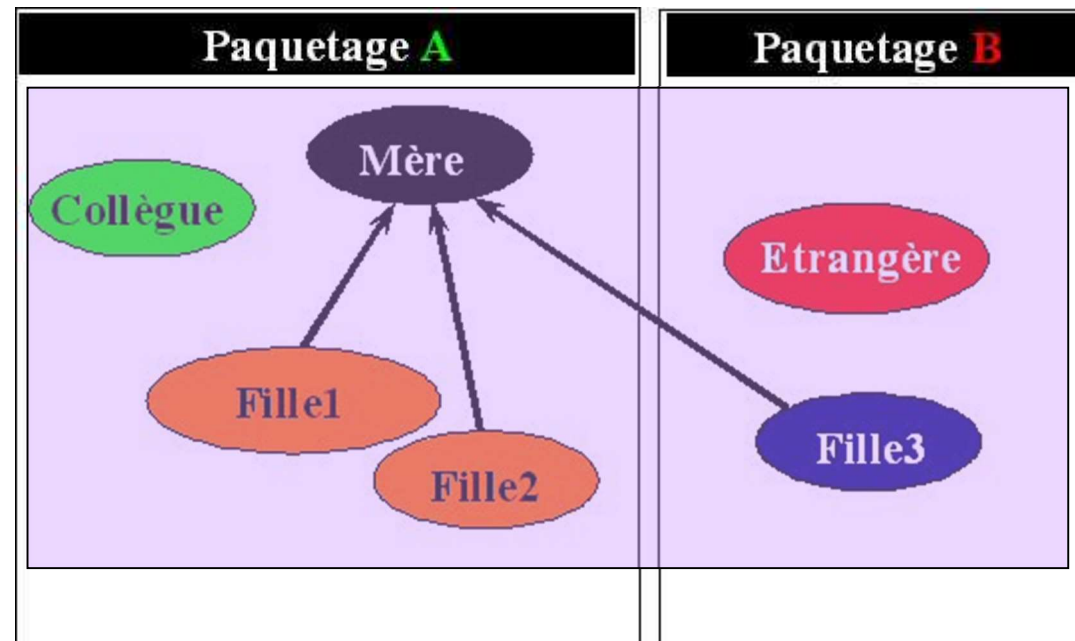
- Les classes **Fille1**, **Fille2** et **Fille3** héritent de la classe **Mère**.
- La classe **Fille3** se trouve dans un paquetage différent de celui des classes **Fille1** et **Fille2**.



4- Classe et Objet en Java

4.2 Packages et protection des données (5/8)

● Attributs ou méthodes **public** :

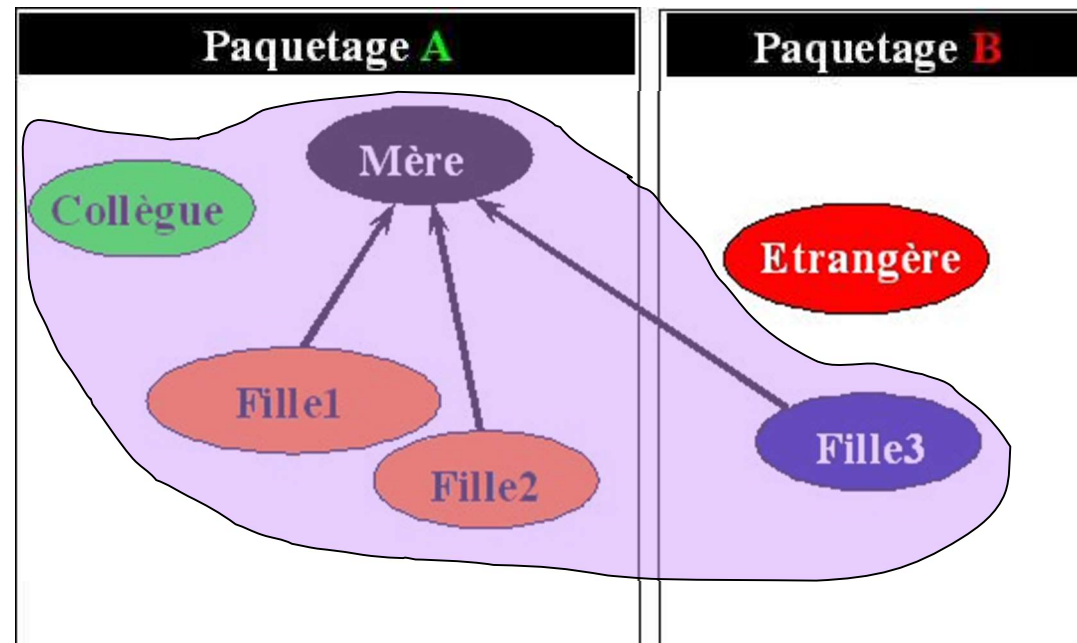


- Les attributs et les méthodes **public** de la classe **Mère** sont accessibles dans toutes les classes.

4- Classe et Objet en Java

4.2 Packages et protection des données (6/8)

● Attributs ou méthodes **protected** :



- Les attributs et les méthodes **protected** de la classe **Mère** sont accessibles dans les classes du même package et dans les classes dérivées.

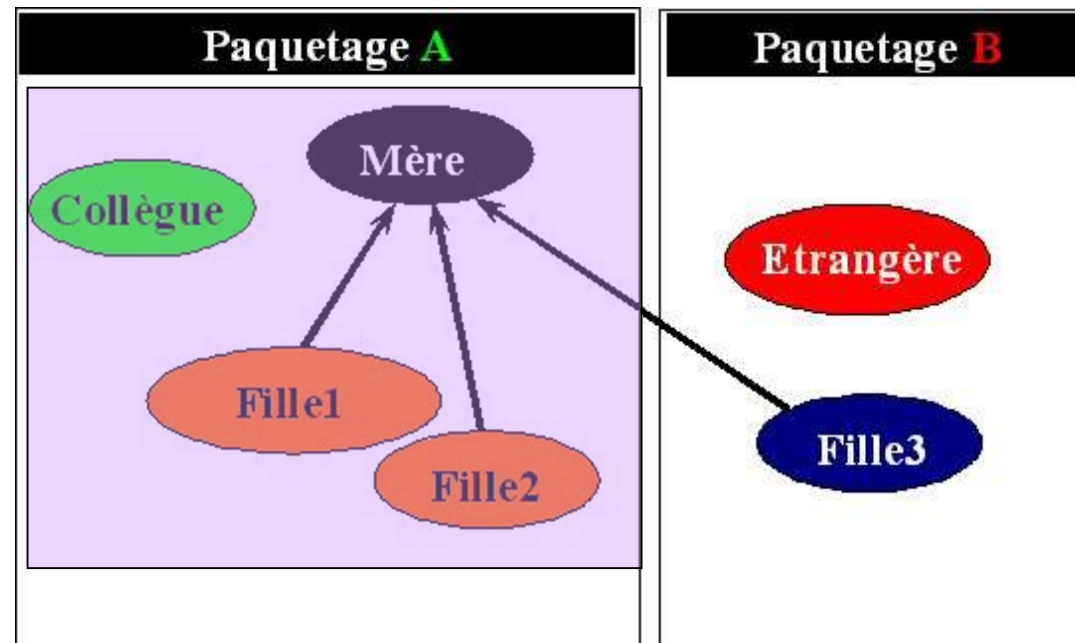
4- Classe et Objet en Java

4.2 Packages et protection des données (7/8)

POO : Java



● Attributs ou méthodes - :



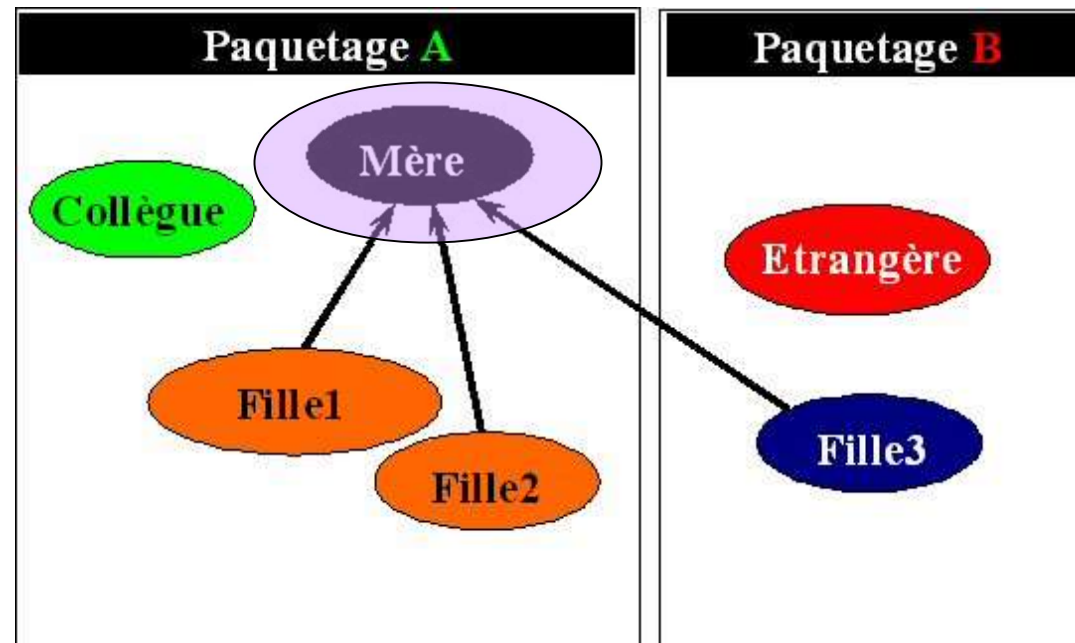
- En l'absence d'un mot clé (**par défaut**), les attributs et les méthodes de la classe **Mère** sont accessibles dans les classes du même package (on parle de **visibilité package private**).



4- Classe et Objet en Java

4.2 Packages et protection des données (8/8)

● Attributs ou méthodes **private** :



- Les attributs et les méthodes **private** de la classe **Mère** ne sont accessibles que dans la classe Mère.

4- Classe et Objet en Java

4.3 Modificateurs de visibilité (1/3)



● Valable pour un attribut ou une méthode :

● « public » :

- accessible partout (dans n'importe quelle classe).

● « private » :

- accessible uniquement depuis sa propre classe.

● « protected » :

- accessible dans les classes du même package et dans les classes dérivées.

● sans mot clé :

- mode de visibilité par défaut : accessible dans toutes les classes du même package



4- Classe et Objet en Java

4.3 Modificateurs de visibilité (2/3)

• « static » :

- attribut ou méthode de classe et non pas d'instance.
- static est cumulable avec d'autres modificateurs.

• « final » :

- Une classe déclarée avec le mot-clé **final** signifie qu'elle ne peut pas être étendue. Autrement dit, aucune classe ne peut hériter d'une classe final.
- Une méthode déclarée avec le mot-clé **final** signifie qu'elle ne peut pas être redéfinie dans une classe dérivée.
- Un attribut déclaré avec le mot-clé **final** signifie que la valeur de cet attribut ne peut plus être modifiée une fois que celle-ci est initialisée.

• **final** int MAX=2; /* déclare une constante d'instance MAX qui vaut 2. */

• **static final** float PI=3.141f; /* déclare la constante de classe PI. */





4- Classe et Objet en Java

4.3 Modificateurs de visibilité (3/3)

Fichier " TestMachin.java "

```
class Machin {  
    private int x;  
    public int y;  
    final int entier =2;  
    Machin ( ) { ... }; // constructeur  
    void modifx(int value) { x=value;}  
    static void truc( ) { ... }; // méthode de classe  
}  
public class TestMachin {  
    public static void main(String[ ] args)  
    { Machin m= new Machin( ); // création d'une instance m de Machin.  
      m.y = 5;  
      m.x = 6;  
      m.entier =4;  
      m.modifx(6);  
      m.truc();  
      Machin.truc();  
    }  
}
```





4- Classe et Objet en Java

4.4 Constructeur d'une classe (1/2)

● Un constructeur est une méthode spéciale d'initialisation d'un objet :

- un constructeur n'a pas de type de retour, et se nomme toujours de la même manière que sa classe
- Une classe peut avoir plusieurs constructeurs qui diffèrent par le nombre et/ou la nature de leurs paramètres (principe de **surcharge**).

● Utilité d'un constructeur :

- un constructeur est une méthode spéciale appelée à la création d'un objet pour initialiser cet objet.
- **surcharger** les constructeurs pour une même classe ce qui permet d'initialiser les objets de diverses manières.

● **NB.** La surcharge peut être appliquée pour n'importe quelle méthode et pas seulement pour les constructeurs.

4- Classe et Objet en Java

4.4 Constructeur d'une classe (2/2)

P.O.O : Java



● Exemple de classe :

Fichier " ExempleDate.java "

```
class ExempleDate {
    private int j;    // jour
    private int m;    // mois
    private int a;    // an
    void initialise(int jour, int mois, int an)
    { j=jour; m=mois; a=an; }

    ExempleDate()                // constructeur sans paramètres
    { j=1; m=1; a=1900; }

    ExempleDate(int jour, int mois, int an) // constructeur avec paramètres
    { j=jour; m=mois; a=an; }

    void affiche( )
    { System.out.println(j+"/"+m+"/"+a); }
}
```





4- Classe et Objet en Java

4.5 Appels de constructeur

Exemples 1:

- Le constructeur **ExempleDate** sans arguments permet d'initialiser une date à 1/1/1900.

```
// création d'une date (1/1/1900)  
ExempleDate d1 = new ExempleDate();
```

Exemples 2:

- Le constructeur **ExempleDate** à 3 arguments permet d'initialiser une date avec des valeurs quelconques.

```
// création d'une date d2 (1/10/2022)  
ExempleDate d2 = new ExempleDate(1,10,2022);
```



Attention :

- Si aucun constructeur n'est spécifié, dans la définition de la classe, un constructeur par défaut vous est obligatoirement fourni par le compilateur, celui-ci n'admet aucun paramètre.





4- Classe et Objet en Java

4.6 Référence implicite : **this** (1/3)

- Le mot clé **this** représente une référence sur l'objet courant.
 - celui qui est en train d'exécuter la méthode (ou l'attribut) dans laquelle se trouvent les instructions concernées.
- **this** peut être utile :
 - Pour déclencher un constructeur depuis un autre constructeur.
 - Pour lever des ambiguïtés entre les noms d'attributs et les noms d'arguments de méthodes (ou constructeurs)
 - Lorsqu'une variable locale (ou un paramètre) "cache", en portant le même nom, un attribut de la classe.

4- Classe et Objet en Java

4.6 Référence implicite : **this** (2/3)

● Exemple d'utilisation de **this** pour lever des ambiguïtés.

Fichier " ExempleDate.java "

```
class ExempleDate {
    private int j;    // jour
    private int m;    // mois
    private int a;    // an

    void initialise(int jour, int mois, int an)
    { j=jour; m=mois; a=an; }

    ExempleDate( )                // constructeur sans paramètres
    { j=1; m=1; a=1900; }

    ExempleDate(int j, int m, int a) // constructeur avec paramètres
    { this.j = j; this.m = m; this.a = a; }

    void affiche( )
    { System.out.println(j+"/"+m+"/"+a); }
}
```



4- Classe et Objet en Java

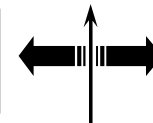
4.6 Référence implicite : **this** (3/3)

- Exemple d'utilisation de **this** pour déclencher un constructeur depuis un autre constructeur.

Fichier " ExempleDate.java "

```
class ExempleDate {
    private int j;    // jour
    private int m;    // mois
    private int a;    // an
    void initialise(int jour, int mois, int an)
    { j=jour; m=mois; a=an; }
```

```
ExempleDate( )
{ j=1; m=1; a=1900; }
```



```
ExempleDate( )
{ this(1,1,1900); }
```

```
ExempleDate(int j, int m, int a) // constructeur avec paramètres
{ this.j = j; this.m = m; this.a = a; }
```

```
void affiche( )
{ System.out.println(j+"/"+m+"/"+a); }
```



4- Classe et Objet en Java

4.7 Méthode **equals()**

P.O.O : **Java**



- La méthode **equals()** permet de comparer deux objets pour vérifier s'ils sont "égaux" selon une définition personnalisée.
 - Signature : **public boolean equals(Object obj)**
- La méthode **equals()**, héritée de la classe **Object**, compare les références (adresse en mémoire) des objets, pas leur contenu.
- Pour une comparaison basée sur le **contenu des objets** (valeurs des attributs), il est courant de **redéfinir** la méthode **equals()** dans vos classes.

@Override

```
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || this.getClass() != obj.getClass()) return false;
    ExempleDate ed = (ExempleDate) obj;
    return this.j == ed.j && this.m == ed.m && this.a == ed.a ;
}
```

4- Classe et Objet en Java

4.8 Méthode **hashCode()**

POO : Java



- La méthode **hashCode()** retourne un code de hachage pour un objet, qui est un entier représentant cet objet de manière unique dans un contexte donné.
 - Signature : **public int hashCode()**
- Le **hashCode** est utilisé principalement dans les structures de données basées sur des tables de hachage (comme **HashMap** et **HashSet**). Il permet de déterminer la position d'un objet dans une table de hachage.
- Si **deux objets sont égaux** (selon **equals()**), ils doivent avoir le **même hashCode()**. Toutefois, deux objets ayant le même **hashCode()** ne sont pas nécessairement égaux.
- Lorsqu'on **redéfinit equals()**, il est obligatoire de **redéfinir également hashCode()** pour respecter le contrat.

@Override

```
public int hashCode() {  
    return Objects.hash(j, m, a);  
}
```

4- Classe et Objet en Java

4.9 Méthode **toString()**



- La méthode **toString()** retourne une représentation sous forme de chaîne de caractères d'un objet.
 - Signature : **public String toString()**
- Par défaut, la méthode **toString()** retourne une chaîne composée du nom de la classe, suivie du symbole @, puis d'un identifiant hexadécimal représentant le code de hachage de l'objet.
- Il est courant de **redéfinir** la méthode **toString()** pour fournir une représentation plus lisible des objets.

@Override

```
public String toString() {  
    return "Date{ jour= " + j + " , mois=" + m + " , annee=" + a + " }";  
}
```

4- Classe et Objet en Java

4.10 Un programme **Java** complet (1/3)

POO : Java



Fichier " ExempleDate.java "

```
import java.util.Objects;
public class ExempleDate {
    private int j;           // jour
    private int m;           // mois
    private int a;           // année
    public ExempleDate( ) { this(1,1,1900); }
    public ExempleDate(int j, int m, int a) { this.j = j; this.m = m; this.a = a; }
    public void affiche( ) { System.out.println(j + "/" + m + "/" + a); }
    public void initialise(int jour, int mois, int an) { j=jour; m=mois; a=an; }
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || this.getClass() != obj.getClass()) return false;
        ExempleDate ed = (ExempleDate) obj;
        return this.j == ed.j && this.m == ed.m && this.a == ed.a ;
    }
    @Override
    public int hashCode() { return Objects.hash(j, m, a); }
    @Override
    public String toString() {
        return "Date{ jour= " + j + " , mois=" + m + " , année=" + a + " }";
    }
}
```



4- Classe et Objet en Java

4.10 Un programme **Java** complet (2/3)

Suite du fichier " ExempleDate.java "

```
java ExempleDate.java 22 4 2010
```

```
public static void main(String[ ] args)
{
    ExempleDate d1 = new ExempleDate(1,10,2022);
    ExempleDate d2= new ExempleDate( );
    ExempleDate d3= new ExempleDate(Integer.parseInt(args[0]),
                                     Integer.parseInt(args[1]), Integer.parseInt(args[2]));
    d2.affiche( );
    d1.initialise(22,10,2025);
    System.out.println(d1);
    if( ! d3.equals(d1))
        System.out.println(d3);
}
}
```

Résultat d'affichage

4- Classe et Objet en Java

4.10 Un programme **Java** complet (3/3)

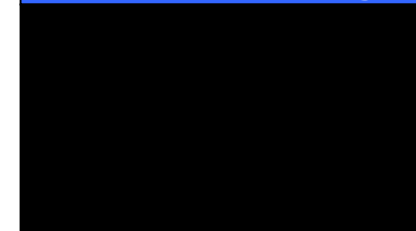
P.O.O : **Java**



- La méthode **main** est le point d'entrée du programme.
- Un **bloc statique** permet d'initialiser les membres statiques lors du chargement de la classe.
 - Il s'exécute **avant** l'appel de la méthode main. Il est possible d'avoir plusieurs blocs statiques, qui s'exécuteront dans le même ordre que celui de leur déclaration dans le programme.

```
class ExempleBlocStatic {
    private static String val; private static int nbre;
    static {
        System.out.println("Dans le bloc 1 static");
        val = "bonjour";
    }
    static {
        System.out.println("Dans le bloc 2 static");
        val = "bonsoir"; nbre = 5;
    }
    public static void main(String[ ] args)
    { System.out.println("Dans main");
      System.out.println("Valeur val : "+val);
      System.out.println("Valeur nbre : "+nbre);
    }
}
```

Résultat d'affichage



4- Classe et Objet en Java

4.11 Création d'instances de classes (1/3)

P.O.O : Java



- Les instances des classes (**objets**) sont systématiquement manipulées à l'aide d'une référence (pointeur sur la zone mémoire représentant l'objet).

- la création d'une variable instance d'une classe nécessite deux opérations :

- 1- la création d'une **référence** qui ne pointe sur rien:

```
ExempleDate d1;  
ExempleDate d2;
```

- 2- la réservation de la place mémoire à l'aide de l'opérateur **new** et l'affectation de la donnée.

```
d1 = new ExempleDate( ); // le constructeur initialise d1 à 1/1/1900.
```

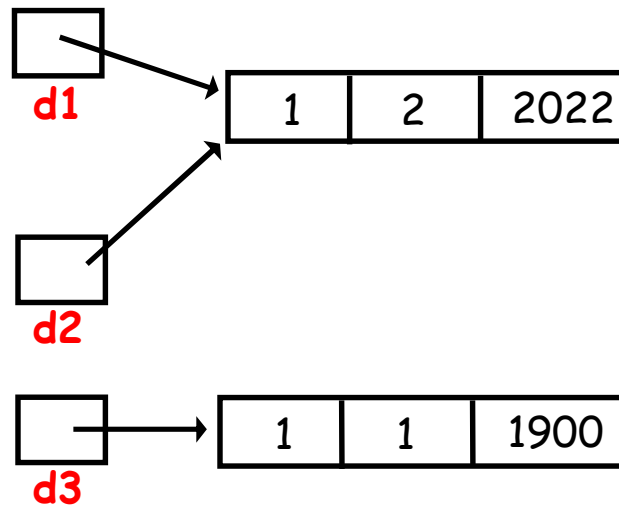
```
d2 = new ExempleDate(15,9,2022); // le constructeur initialise d2 à 15/9/2022.
```



4- Classe et Objet en Java

4.11 Création d'instances de classes (2/3)

Schéma d'implantation en mémoire :



ExempleDate **d1**;

d1 = new ExempleDate(1,2,2022);

ExempleDate **d2**;

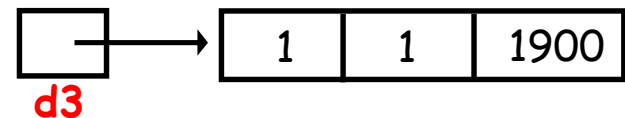
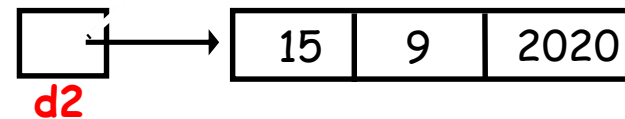
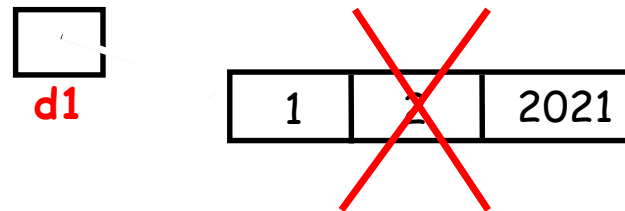
d2 = **d1**;

ExempleDate **d3** = new ExempleDate();

4- Classe et Objet en Java

4.11 Création d'instances de classes (3/3)

● Schéma de libération de mémoire :



ExempleDate **d1**;

d1 = new ExempleDate(1,2,2021);

ExempleDate **d2**;

d2 = **d1**;

ExempleDate **d3** = new ExempleDate();

d1 = null;

d2 = new ExempleDate(15,9,2020);





4- Classe et Objet en Java

4.12 Destruction d'un objet (1/5)

- La plupart des langages orientés objets possèdent une méthode nommée **destructeur**, appelée pour détruire un objet.
- La destruction d'un objet consiste à libérer les ressources qui lui ont été déjà affectées par le constructeur.
 - Un objet est détruit lorsqu'il ne référence aucune zone mémoire.
- En **Java**, la destruction des objets est prise en charge par un mécanisme automatique appelé **ramasse-miettes** (ou **garbage collector GC**).
- Le **GC** fonctionne en arrière-plan pendant l'exécution d'un programme Java. il est géré dans un **thread** de faible priorité.
 - difficile de savoir à quel moment le **GC** détruira réellement votre objet.





4- Classe et Objet en Java

4.12 Destruction d'un objet (2/5)

- La récupération de mémoire peut être aussi invoquée explicitement par le programmeur à des moments bien précis avec la commande **System.gc()**.
- Certaines situations nécessitent un nettoyage spécial que le **GC** ne peut pas effectuer :
 - Par exemple, certains fichiers ont été ouverts pendant la durée de vie de l'objet et vous voulez vérifier qu'ils sont correctement fermés quand l'objet est détruit.
- Une autre méthode spéciale, appelée **finalize()**, peut être définie. Cette méthode, **dépréciée** depuis **Java 9**, est appelée (si elle est présente) par le **GC** pour effectuer un nettoyage spécial lorsqu'un objet est sur le point d'être collecté.
- Depuis **Java 18**, il est possible de désactiver la finalisation pour une application à l'exécution via l'option JVM :
--finalization=disabled.



4- Classe et Objet en Java

4.12 Destruction d'un objet (3/5)

POO : Java



Fichier " DateGC1.java "

```
class DateGC1 {
    private int j;
    private int m;
    private int a;
    void initialise(int jour, int mois, int an) { j=jour; m=mois; a=an; }
    DateGC1( ) { j=1; m=1; a=1900; }
    DateGC1(int j, int m, int a) { this.j = j; this.m = m; this.a = a; }
    void affiche( ) { System.out.println(j+"/"+m+"/"+a); }
    protected void finalize() { System.out.println("Nettoyage spéciale"); }
    public static void main(String[ ] args)
    { DateGC1 d = new DateGC1(1,10,2024);
      d.affiche( );
      d=null; // d ne désigne plus rien. L'objet n'est plus accessible.
              // A tout moment le GC peut détruire l'objet.
      System.gc( ); // appel implicite du GC
    }
}
```

Résultat d'affichage



4- Classe et Objet en Java

4.12 Destruction d'un objet (4/5)

Fichier " DateGC2.java "

```
class DateGC2 {  
    private int j;  
    private int m;  
    private int a;  
    void initialise(int jour, int mois, int an) { j=jour; m=mois; a=an; }  
    DateGC2( ) { j=1; m=1; a=1900; }  
    DateGC2(int j, int m, int a) { this.j = j; this.m = m; this.a = a; }  
    void affiche( ) { System.out.println(j+"/"+m+"/"+a); }  
    protected void finalize() { System.out.println("Nettoyage spéciale"); }  
    public static void main(String[ ] args)  
    {  
        DateGC2 d = new DateGC2(1,10,2024);  
        d.affiche( );  
        DateGC2 d2=d;  
        d=null; // d ne désigne plus rien  
        System.gc( ); // appel implicite du GC  
    }  
}
```

Résultat d'affichage





4- Classe et Objet en Java

4.12 Destruction d'un objet (5/5)

Fichier " DateGC3.java "

```
class DateGC3 {  
    private int j;  
    private int m;  
    private int a;  
    void initialise(int jour, int mois, int an) { j=jour; m=mois; a=an; }  
    DateGC3() { j=1; m=1; a=1900; }  
    DateGC3(int j, int m, int a) { this.j = j; this.m = m; this.a = a; }  
    void affiche() { System.out.println(j+"/"+m+"/"+a); }  
    protected void finalize() { System.out.println("Nettoyage spéciale"); }  
    public static void main(String[ ] args)  
    {  
        DateGC3 d = new DateGC3(1,10,2024);  
        d.affiche();  
        DateGC3 d2=d;  
        d=null;    d2=null;  
        System.gc(); // appel implicite du GC  
    }  
}
```

Résultat d'affichage



4- Classe et Objet en Java

4.13 Comparaison d'instances (1/4)



● Utilisation de == :

- Il y a égalité que si d1 et d2 désignent la même zone mémoire représentant une date.

```
ExempleDate d1 = new ExempleDate(1,1,2022);
ExempleDate d2 = new ExempleDate(1,1,2022);
System.out.println(d1 == d2)      // false
```

```
ExempleDate d1 = new ExempleDate(1,1,2022);
ExempleDate d2 = d1;
System.out.println(d1 == d2)      // true
```

● Pour comparer deux instances, il faut utiliser la méthode equals() :

```
ExempleDate d1 = new ExempleDate(1,1,2020);
ExempleDate d2 = new ExempleDate(1,1,2020);
if(d1.equals(d2))
    //égalité si « equals » est redéfinie.
```



4- Classe et Objet en Java

4.13 Comparaison d'instances (2/4)

Fichier "CacheStringWrappers .java "

```
public class CacheStringWrappers {  
    public static void main(String[] args) {  
        String s1 = "cpi2";  
        String s2 = "cpi2";  
  
        System.out.println("String littérales :");  
        System.out.println(" s1 == s2 ? " + (s1 == s2));  
        System.out.println(" s1.equals(s2) ? " + s1.equals(s2));  
  
        String s3 = new String("cpi2");  
        System.out.println("String via constructeur :");  
        System.out.println(" s1 == s3 ? " + (s1 == s3));  
        System.out.println(" s1.equals(s3) ? " + s1.equals(s3));  
    }  
}
```



4- Classe et Objet en Java

4.13 Comparaison d'instances (3/4)

Fichier "CacheStringWrappers.java"

```
Integer i1 = 127;
Integer i2 = 127;
Integer i3 = 128;
Integer i4 = 128;

System.out.println("Wrappers (Integer) :");
System.out.println(" i1 == i2 ? " + (i1 == i2));
System.out.println(" i1.equals(i2) ? " + i1.equals(i2));
System.out.println(" i3 == i4 ? " + (i3 == i4));
System.out.println(" i3.equals(i4) ? " + i3.equals(i4));

Integer n1 = new Integer(127);
Integer n2 = new Integer(127);
System.out.println("Wrappers via constructeur :");
System.out.println(" n1 == n2 ? " + (n1 == n2));
System.out.println(" n1.equals(n2) ? " + n1.equals(n2));
```

```
}
```



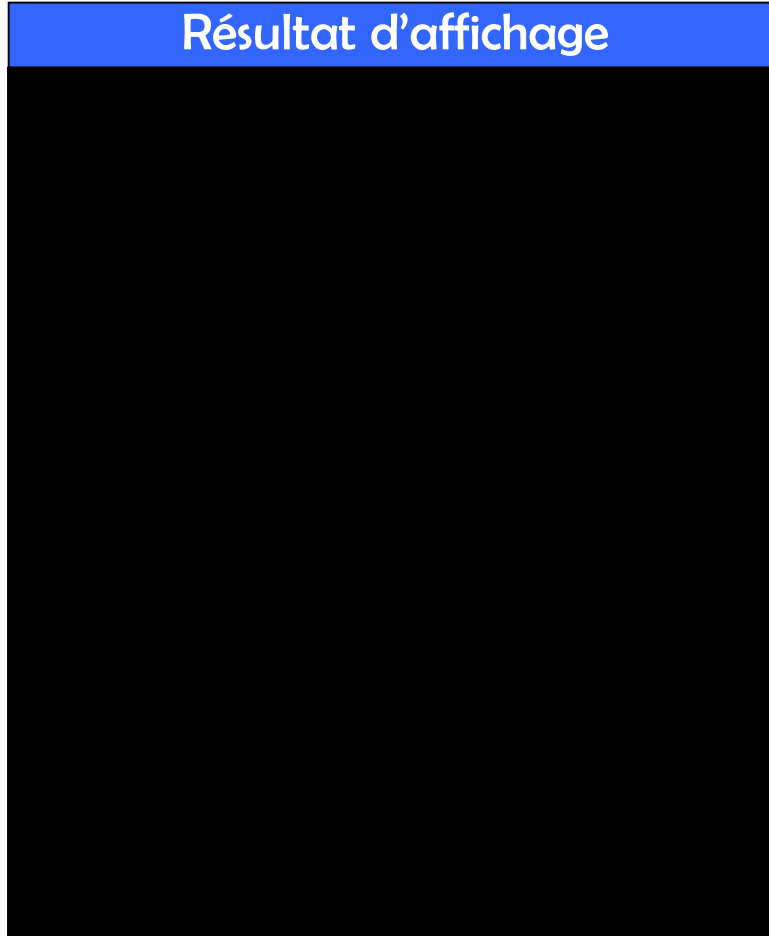
4- Classe et Objet en Java

4.13 Comparaison d'instances (4/4)

P00 : Java



Résultat d'affichage





4- Classe et Objet en Java

4.14 Opérateurs sur les références

● Les seuls opérateurs sur les références sont des opérateurs logiques :

- **==** : permet de tester si deux références désignent le même objet.
- **!=** : permet de tester si deux références ne désignent pas le même objet.
- **equals** : permet de comparer le contenu de deux références.
- **instanceof** : permet de tester si l'objet référencé est une instance d'une classe donnée.

```
if ( d1 instanceof ExempleDate ) {  
    ... Traitement  
}
```



4- Classe et Objet en Java

4.15 Accès aux membres (1/3)

- **Les membres sont accessibles via une **instance** de la classe ou via la **classe** (pour les membres **statiques**).**
 - On distingue 4 types d'accès aux membres d'une classe :
 - On accède à un attribut d'instance.
 - On invoque la méthode d'instance.
 - On accède à un attribut de classe.
 - On invoque une méthode de classe.
- **L'accessibilité des membres est pondérée par les modificateurs de visibilité (public, private, ...).**



4- Classe et Objet en Java

4.15 Accès aux membres (2/3)

P.O.O : Java



Exemple :

```
class Cercle
{
    private double r;    // rayon du cercle
    public Cercle(double r)
    {
        this.r = r; // On accède à l'attribut 'r' de l'instance en cours
    }
    public Cercle( )
    {
        this(2.0); // On invoque le premier constructeur
    }
    public double pi()
    {
        return Math.PI; // On accède à l'attribut statique 'PI' de la classe 'Math'
    }
    public double area()
    {
        return pi() * Math.pow(r,2); // On invoque la méthode statique 'pow'
                                     // de la classe 'Math'
    }
}
```



4- Classe et Objet en Java

4.15 Accès aux membres (3/3)

POO : Java



● Déclaration d'une méthode avec un nombre indéterminé de paramètres (**varargs**)

Fichier " TestVarargs.java "	Résultat d'affichage
<pre> public class TestVarargs { static int somme(int . . . tab) { int res = 0; for (int elt : tab) res += elt; return res; } public static void main(String[] args) { System.out.println(somme(2)); System.out.println(somme(2, 3)); System.out.println(somme(2, 3, 4)); System.out.println(somme(2, 3, 4, 5)); } } </pre>	

- **varargs** est disponible depuis **Java 5**.
- On ne peut avoir qu'un seul varargs par méthode.
- Le paramètre varargs doit être le dernier dans la liste de paramètres d'une méthode.

4- Classe et Objet en Java

4.16 Mode de passage des paramètres (1/5)

● Le mode de passage des paramètres dans les méthodes dépend de la nature des paramètres :

- par référence pour les objets,
- par copie pour les types primitifs.

Résultat d'affichage

Fichier "TestParam.java"

```
public class TestParam {
    static void modifier(int value, StringBuffer chaine) {
        value++;
        chaine.append(" tout le monde");
    }

    public static void main(String[ ] args) {
        int a=0;
        StringBuffer chaine= new StringBuffer("Bonjour" );
        modifier(a, chaine);
        System.out.println(" La valeur de a = "+a);
        System.out.println(" La valeur de la chaine = "+chaine);
    }
}
```

P.O.O : Java





4- Classe et Objet en Java

4.16 Mode de passage des paramètres (2/5)

POO : Java



Fichier "TestPassage.java"

```
class Personne {  
    String nom;  
    Personne(String nom) { this.nom = nom; }  
}  
public class TestPassage {  
    static void modifierValeur(int val) {  
        val = 20;  
        System.out.println("Dans la méthode modifierValeur() : " + val);  
    }  
    static void modifierNom(Personne p) {  
        p.nom = "Ali";  
        System.out.println("Dans la méthode modifierNom() : " + p.nom);  
    }  
    static void changerReference(Personne p) {  
        p = new Personne("Ali");  
        System.out.println("Dans la méthode changerReference() : " + p.nom);  
    }  
}
```



4- Classe et Objet en Java

4.16 Mode de passage des paramètres (3/5)

P.O.O : Java



Fichier "TestPassage .java "

```
public static void main(String[] args) {  
    int nombre = 10;  
    System.out.println("Avant l'appel : " + nombre);  
    modifierValeur(nombre);  
    System.out.println("Après l'appel : " + nombre);  
    System.out.println("*****");  
    Personne personne1 = new Personne("Salma");  
    System.out.println("Avant l'appel : " + personne1.nom);  
    modifierNom(personne1);  
    System.out.println("Après l'appel : " + personne1.nom);  
    System.out.println("*****");  
    Personne personne2 = new Personne("Salma");  
    System.out.println("Avant l'appel : " + personne2.nom);  
    changerReference(personne2);  
    System.out.println("Après l'appel : " + personne2.nom);  
}
```



4- Classe et Objet en Java

4.16 Mode de passage des paramètres (4/5)

P.O.O : Java



Résultat d'affichage





4- Classe et Objet en Java

4.16 Mode de passage des paramètres (5/5)

P.O.O : Java



```
class Boxing7{
    static void m(long i, long j){System.out.println("long long");}
    static void m(short... i){System.out.println("short...");}
    public static void main(String args[]){
        short s1=30, s2=40;
        m(s1,s2);
    }
}
```

Résultat d'affichage

```
class Boxing8{
    static void m(long i){System.out.println("long");}
    static void m(Integer... i){System.out.println("Integer...");}
    public static void main(String args[]){
        Integer a=30;
        m(a);
    } }
```

Résultat d'affichage

4- Classe et Objet en Java

4.17 Records (1/5)

P.O.O : Java



- **Les records sont des nouveaux types d'objets immuables en Java. Ce concept est apparu à partir de Java 16.**
 - Avant l'introduction des records, pour créer des objets immuables en Java, il fallait définir une classe avec des attributs privés, exposer uniquement des accesseurs (getters) et **ne pas inclure** de mutateurs (setters).
 - Un record est essentiellement une classe qui permet de regrouper des valeurs (ou composants) tout en garantissant leur immutabilité. Cela signifie qu'une fois qu'un record est créé, ses valeurs ne peuvent pas être modifiées.
- **Le concept de record en Java permet aux développeurs de créer des structures de données immuables avec une syntaxe simple et concise, réduisant ainsi la complexité du code tout en améliorant sa lisibilité.**

4- Classe et Objet en Java

4.17 Records (2/5)

P.O.O : Java



- Pour mieux comprendre ce concept, considérons l'exemple de la classe Point suivante :

```
import java.util.Objects;
public final class Point {
    private final double x; private final double y;
    public Point( double x, double y ) { this.x = x; this.y = y; }
    public double getX() { return x; } public double getY() { return y; }
    @Override
    public boolean equals( Object object ) {
        if ( this == object ) { return true; }
        else if ( object instanceof Point ) {
            Point other = (Point) object; return x == other.x && y == other.y; }
        else { return false; }
    }
    @Override
    public int hashCode() { return Objects.hash(x, y); }
    @Override
    public String toString() {
        return "Point [x=" + x + ", y=" + y + "]"; }
}
```

- Il est possible de définir l'**équivalent** de cette classe en une seule ligne de code :

```
public record Point (double x, double y) {}
```

4- Classe et Objet en Java

4.17 Records (3/5)

POO : Java



- Dans cet exemple, Point est un record avec deux composants : x et y. Le compilateur génère **automatiquement** :
 - Deux attributs **x** et **y** privés et immuables (final)
 - Un **constructeur canonique** à deux paramètres qui prend tous les paramètres correspondant aux attributs définis dans le record.
 - Pour chaque attribut, une **méthode d'accès en lecture**. Mais **attention**, ces méthodes ne sont pas préfixées par get. Par exemple, pour l'attribut nommé x, la méthode d'accès sera simplement appelée **x()** au lieu de getX().
 - Les méthodes **equals()**, **hashCode()** et **toString()** sont générées automatiquement et peuvent être redéfinies si nécessaire.
- Un record est une **classe final**, ce qui signifie qu'il ne peut pas être étendu par héritage, que ce soit par un autre record ou une autre classe.
- Un record est conçu pour être une classe de données immuable et concise. Il hérite automatiquement de la classe java.lang.Record, ce qui interdit de spécifier une autre classe parente via extends dans la déclaration du record.

4- Classe et Objet en Java

4.17 Records (4/5)

POO : Java



- Il est tout à fait possible d'utiliser un type record de la même manière qu'une classe traditionnelle.

Résultat d'affichage

```
public class TestPoint {
    public static void main( String[] args ) {
        Point p1 = new Point( 2.5, 3.9 );
        Point p2 = new Point( 2.5, 3.9 );
        System.out.println( p1 );
        System.out.println( "Egalité : " + p1.equals(p2) );
        System.out.println( "Getters : " + p2.x() + " - " + p2.y() );
    }
}
```

- Il est possible d'enrichir un record avec des membres supplémentaires tels que des attributs statiques, des méthodes et des constructeurs personnalisés.

- Lorsqu'un record définit un **constructeur non canonique**, ce dernier doit **obligatoirement** appeler un autre constructeur (le plus souvent le constructeur canonique généré automatiquement) en tant que **première instruction**.





4- Classe et Objet en Java

4.17 Records (5/5)

- Il est possible d'utiliser un **constructeur compact** pour définir des traitements associés au constructeur canonique généré automatiquement. Ces traitements peuvent inclure, par exemple, la validation des valeurs fournies. Le constructeur compact simplifie la syntaxe et permet d'intégrer facilement des validations lors de la création d'instances de records. Il ne nécessite pas de spécifier les paramètres ; il utilise ceux qui sont définis dans l'en-tête du record. Il est ainsi possible de modifier ces paramètres avant de les passer au constructeur canonique.

```
record Point(double x, double y) {  
    // Constructeur compact  
    public Point{  
        if (x < 0) {  
            System.out.println(x+" doit être positif");  
            x=0;  
        }  
        if (y < 0) {  
            System.out.println(y+" doit être positif");  
            y=0;  
        }  
    }  
    // Constructeur non canonique  
    public Point() { this( 0, 0 ); }  
}
```

